Operating Systems 2023 Spring Final Exam (2023/05/31)

1. (10%) How does the deadlock for Maekawa's mutual exclusion Algorithm happen? Please give an example to demonstrate it and provide a solution to avoid this.
   Ans.
   Deadlock example (4%):
   Consider three processes, p1, p2, and p3, where V1={p1, p2}, V2={p2, p3}, and V3={p3, p1}. If they concurrently request to enter the CS, then it is possible for p1 to reply to itself and hold off p2, for p2 to reply to itself and hold off p3 , and for p3 to reply to itself and hold off p1. Each process has received only one replies and none can proceed.
   Solution to avoid deadlock (6%):
   Deadlock handling requires three extra types of messages:
   A FAILED message from pi to pj indicates that pi cannot grant pj a vote because it has voted to a process with a higher priority request.
   An INQUIRE message from pi to pj indicates that pi would like to find out from pj if it has all votes from processes in Vj.
   A YIELD message from pi to pj indicates that pi is returning the vote to pj for yielding to a higher priority request at pj.
   To avoid deadlock, a process p needs to yield a vote if the timestamp of p's request is larger than the timestamp of some other requests waiting for the same vote.
   The inquires and yield process need to be added. When the process has granted some other lower priority process and the incoming higher priority process comes, the inquiry is sent to the granted process. The granted process checks the existence of received failed and the non-replied yield. It sends yield back to the deciding process and let the process put it into the queue and proceed the incoming process.

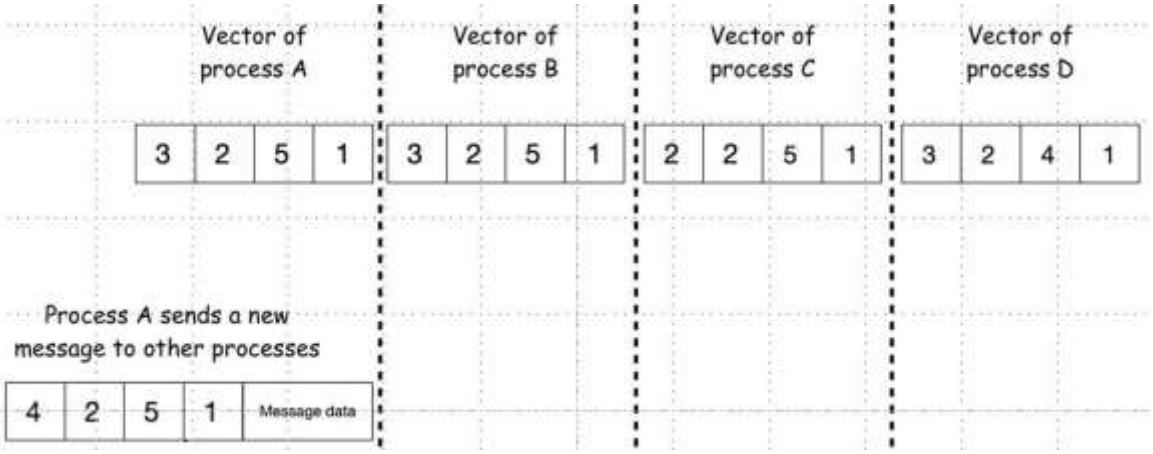
2. (6%) Please give three possible cache location policies and their advantages.
   Ans. (2% each)
   Server memory: easy implementation and transparent to clients.
   Client memory: provide good performance speed up.
   Client disk: can cache more and larger files.

3. (10%) Please describe the pros and cons of blocking and non-blocking message passing.
   Ans.
   A. blocking primitives:
   Pros:(1%)

The blocking primitives are much easy to implement.
Cons:(3%)
The sender and/or the receiver can block forever if the other party crashes or messages are lost.
The concurrency degree between sender and receiver is limited.
The communication deadlock is more likely to happen.
B. non-blocking primitives:
Pros (2%):
The sender and/or the receiver can continue their executions without waiting for each other.
The system is less likely to run into a deadlock
Cons (4%):
The sender does not know whether or not the receiver has actually received the message.
The sender cannot modify the message buffer until the message has been sent.
Additional overhead is required to notify the sender and the receiver.
Additional data copy is required to reuse message buffer.

4. (4%) What is the buffered and un-buffered multicasting?
   Ans. (2% each)
   Un-buffered multicasts are received only by processes that are ready to receive the message.
   Messages are buffered for buffered multicast, so all processes eventually receive the message.

5. (6%) For the figure below, which processes can receive the new message sent from process A in the CBCAST protocol? Why?

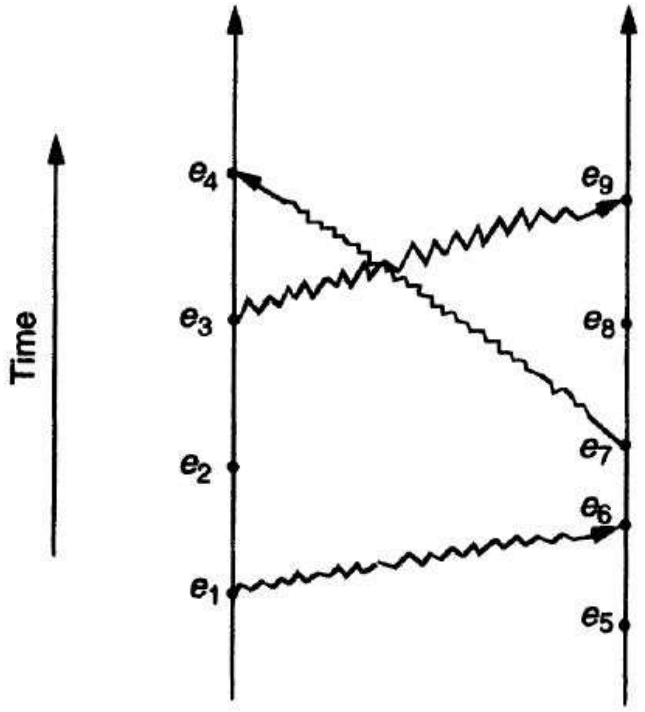


   Ans. (2% each)
   In CBCAST, If S[i]=R[i]+1 and for all $j \neq i$, $S[j] \leq R[j]$, then the message is delivered to the application, otherwise, the message is kept in the buffer. Process B can

receive the message sent from process A because A[1]=B[1]+1 and A[2]=B[2], A[3]=B[3], A[4]=B[4].
Process C can't receive the message because the condition A[1] != C[1]+1.
Process D can't receive the message because the condition A[3] > D[3].

6. (6%) Please give 3 differences between replication and caching.
   Ans. (2% each)
   1. A replica is a copy in a server disk and a cache is a copy of a replica.
   2. A cache is dependent on the locality in file access patterns.
   3. A replica depends on availability and performance requirements.
   4. A replica is usually persistent and widely known.
   5. The existence of a cache depends on the presence of a replica.

7. (11%) Using the space-time diagram of the figure below, please list all pairs of concurrent events according to the happened-before relation.



   Ans. (1% each)
   (e1 e5) (e2 e5) (e2 e6) (e2 e7) (e2 e8) (e3 e5) (e3 e6) (e3 e7) (e3 e8)
   (e4 e8) (e4 e9)

8. (10%) What are the design characteristics of AFS for achieving high scalability? Please briefly describe these characteristics?
   Ans.
   AFS has two unusual design characteristics to achieving high scalability.
   1. Whole file serving (4%): The entire contents of directories and files are transmitted to client computers by AFS servers. In AFS 3, files larger than 64 Kbytes are transferred in 64 Kbyte chunks.
   2. Whole file caching (6%): Once a copy of a file has been transferred to a client computer it is stored in a cache on the local disk. The cache contains several hundred of the files most recently used on the client computer. The cache is permanent and can survive reboots of the client computer. Local copies of files are used to satisfy clients' open requests in preference to remote copies whenever possible.

9. (6%) Please give and describe at least 3 file sharing sematics of distributed file systems.
   Ans. (2% each)
   (a) One copy update semantics (UNIX): the result of every write operation must be visible immediately.
   (b) Session update semantics (AFS): only after closing the updated file, the result of write operations becomes available.
   (c) Append only update semantics (HDFS): data is never modified once it has been written to a file, and new data can only be appended to the existing file.
   (d) Copy on write update semantics: In a copy-on-write file system, when a process requests to modify a shared file, a copy of the original file is created for the requester if another process has already modified or holds a reference to it. Otherwise, the original file is considered immutable, and the requesting process can safely modify the copy without affecting other processes.

10. (3%) In the central server algorithm for mutual exclusion, describe a situation in which two requests are not processed in happened-before order.
    Ans.
    Process *A* sends a request Ra for entry then sends a message *m* to B. On receipt of m, *B* sends request Rb for entry. To satisfy happened-before order, Ra should be granted before Rb. However, due to the vagaries of message propagation delay, Rb arrives at the server before Ra, and they are serviced in the opposite order.

11. (6%) Discuss whether the following operations are idempotent:

(a) Pressing an elevator call button
(b) Sorting a list
(c) Appending to a file
Ans. (2% each)
(a) Idempotent (elevator stays in requested state)
(b) Idempotent (sorting a sorted list does not change the sorted list)
(c) Non-idempotent (two appends will add twice as much data)

12. (6%) client A has a time of 5:05, client B has a time 5:15, and server S has a time of 5:25. Using the Berkeley algorithm, what are the clock adjustment values sent from S to A, B, and itself, respectively?
    Ans. (2% each)
    (a) on receiving 5:25 from S, A sends -20 to S and B sends -10 to S while S sends 0 to itself.
    (b) Since (-20-10+0)/3=-10, S sends 10 (-10+20=10) to A, sends 0 (-10+10=0) to B, and sends -10 (-10-0=-10) to itself.

13. (8%) There are two kinds of physical clock synchronization.
    (a) What is External synchronization?
    (b) What is Internal synchronization?
    (c) Is externally synchronized clocks also internally synchronized?
    (d) Is internally synchronized clocks also externally synchronized?
    Ans. (2% each)
    (a) External synchronization: Synchronized to a UTC source.
    (b) Internal synchronization: Synchronized within the DS.
    (c) Yes, externally synchronized clocks are also internally synchronized.
    (d) No, internally synchronized clocks are not necessarily externally synchronized.

14. (6%) In algorithms for Distributed Mutual Exclusion (DME), what are the essential requirements for mutual exclusion?
    Ans. (2% each)
    ME1: (safety) At most one process may execute in the CS at a time.
    ME2: (liveness) Requests to enter and exit the CS eventually succeed.
    ME3: (HB ordering) Entry to the CS is granted in the HB order of requests.

15. (3%) Please give 3 possible options of client-initiated cache validation policy.
    Ans. (1% each)
    (1) Checking before every access
    (2) periodic checking
    (3) checking when opens a file

16. Please give and describe at least 3 multi-copy update policies.

    Ans.

    (a) (2%) Read Only Replication (ROR). Only immutable files are replicated to avoid consistency issue.

    (b) (3%) Read One Write All Protocol (R1Wn). A read OP is performed by reading any copy of the file. A write OP is performed by writing to all replicas of the file. Some form of locking has to be used to carry out a write operation. It is suitable for implementing UNIX like semantics. It is very efficient when the read/write ratio is large.

    (c) (3%) Available Copies Protocol. A write OP is performed by writing to all available copies. On recovery from a failure, a server must 1st brings itself up to date by copying from other servers before accepting any user request. It can tolerate site crashes.

    (d) (3%) Primary Copy Protocol. For each replicated file, one copy is designated as the primary copy and all the others are secondary copies. Read OPs can be performed using any copy. All write OPs are 1st performed only on the primary copy. Secondary copies are then updated later.

    (e) (5%) Quorum Based Protocols. Assume that a replicated file F has a total of n copies. For each read OP, a set of r replicas (read quorum) must be contacted to get the latest version. For each write OP, a set of w replicas (write quorum) must be locked to perform the updates. If ($r+w > n$ and $2w>n$) then every read OP always gets the result of the latest write and every write OP is always performed on top of the latest write.